Group 9 — Character Error Rate (CER)

Research report: measuring character-level differences in text produced by Large Language Models

Advanced Programming · Group Discussion Assignment: *Evaluation Methods for Large Language Models (LLMs)* [1]

Lecturer: Mr. Vann Visal

Group member	Student ID
PHYRUN Pichchhorda	e20220895
NOB Sreynich	e20220741
SOUR Alya	e20220969
HOUN Sopanha	e20220209
CHOUB Botumraksa	e20221709
CHHO Sengmeng	e20220296
HEAM Visal	e20221349

Date of submission: 13 November 2026

Companion file: `CER_Evaluation.ipynb` (Python code, collected answers, and saved outputs) [7]

1. Introduction

1.1 What CER measures

Character Error Rate (CER) compares two texts character by character. The first text is the **reference** (the correct text that we expect). The second text is the **prediction** (the text produced by a model). CER counts the smallest number of character changes needed to make the two texts match, and then divides that number by the length of the reference.

A short example: the reference is `cat` and the prediction is `cut`. One character changed (`a` became `u`), and the reference has 3 characters. The CER is $1/3 \approx 0.3333$, or about **33.33%**.

1.2 Why CER matters for evaluating LLMs

Large Language Models (LLMs) (computer programs trained on very large amounts of text so that they can read and write language) are used for text generation, translation, question answering, and many other tasks. Our assignment explains that measuring their performance requires suitable evaluation methods and metrics [1]. Group 9 was assigned CER, which measures character-level substitution, deletion, and insertion errors [1].

CER is useful when the **exact form** of the text matters. A word-level measure treats a word with one wrong letter as one wrong word. CER shows more detail: it can tell us that only one character inside that word was wrong.

1.3 Suitable applications

- **Reading text from images** (often called OCR, optical character recognition), where a single wrong letter changes the stored text [7].
- **Turning speech into written text** (automatic speech recognition). The JiWER library, which we use later, is a Python package for evaluating speech recognition systems, and CER is one of its measures [4].
- **Spelling correction**, the task used in our experiment (Section 6). Edit distance, the idea behind CER, was described with automatic spelling correction as a possible application [6].
- **Any task where a model must copy or rewrite text exactly**, such as names or short fixed sentences.

CER is **less suitable** for open-ended answers, summaries, or explanations, because many different wordings can be equally correct (see Section 9).

2. Theory

2.1 Reference and prediction (hypothesis)

Term	Basic meaning	Example
Reference	The correct text that we expect	`cat`
Prediction (also called **hypothesis**)	The text produced by the model	`cut`

The JiWER documentation uses the word *hypothesis* for the prediction [2]. In this report both words mean the same thing. CER is not symmetric: the length used for division always comes from the **reference**.

2.2 Matches, substitutions, deletions, and insertions

When we line up the two texts (this lining up is called an **alignment**), each step is one of four outcomes:

Outcome	Letter	Meaning	Cost	Reference → prediction
Match (hit)	H	The two characters are the same	0	`cat` → `cat`
Substitution	S	One character is replaced by another	1	`cat` → `cut`
Deletion	D	A reference character is missing from the prediction	1	`cat` → `ct`
Insertion	I	An extra character appears in the prediction	1	`cat` → `cart`

JiWER calls matches **hits** [2].

> **Note on direction.** In this report, a deletion is a reference character that is missing from the prediction, and an insertion is an extra character in the prediction. JiWER's documentation words these counts as changes that turn the hypothesis into the reference [2]. However, the counts that JiWER returned in our checks follow the meaning used here. For example, `plant` → `blat` gave $D = 1$ and $I = 0$ (Section 3.1).

2.3 Minimum character edit distance (Levenshtein distance)

There are many possible ways to change one text into another. CER uses the way that needs the **smallest total number of changes**. This smallest number is called the **minimum edit distance**, or **Levenshtein distance**. JiWER describes its character calculation as a character-level Levenshtein distance [2]. Wagner and Fischer called this the *string-to-string correction problem* (a string is a piece of text): find the minimum number of single-symbol changes, deletions, and insertions that turns one string into another [6].

Why the minimum matters. Suppose we compare `cat` and `cart` position by position. `c` = `c` and `a` = `a`. Then `t` ≠ `r`, and one extra `t` remains, which gives 2 differences. The minimum, however, is only **1** change: insert `r`. One inserted character shifts every later character, so a simple position-by-position comparison can overcount errors.

Sometimes more than one alignment has the same lowest cost. The separate S, D, and I counts can then differ between those alignments, but the total and the CER stay the same [7].

2.4 The CER formula

$$\text{CER} = \frac{S + D + I}{N}$$

- S = number of substitutions
- D = number of deletions
- I = number of insertions
- N = number of characters in the **reference**, including spaces and punctuation under our rules (Section 4)

Matches cost nothing, so they do not appear in the top part of the fraction. Every reference character is matched, substituted, or deleted. Therefore, if H is the number of matches (hits), $N = H + S + D$. Insertions are **not** part of the reference, so they can make the top of the fraction larger than N.

2.5 Ratio versus percentage

The formula gives a **ratio** (a number such as 0.4). To show it as a **percentage**, multiply by 100:

$$0.4 \times 100 = 40\%$$

A stored value of `0.4` means **40%**, not 0.4%. Our Python code stores the ratio and prints it as a percentage with two decimal places using the format `:.2%` [7].

2.6 How to read a CER score

- **Lower is better.** 0% means that the prediction matches the reference exactly under the chosen text rules. A score of 0% does not prove that the reference itself is factually correct.
- **An empty prediction gives 100%.** Every reference character is deleted, for example `cat` → empty text: $3/3 = 100\%$.
- **CER can exceed 100%.** Reference `a` and prediction `abcd` need three insertions. The reference has one character, so $\text{CER} = 3/1 = 300\%$. CER is an error ratio, not a probability, and it is not a share of a fixed total.

2.7 Combining several texts into one overall score

When a model answers several tasks, we add all errors and all reference lengths **before** dividing:

$$\text{Overall CER} = \frac{\sum(S + D + I)}{\sum N}$$

This is generally **not** the same as a plain average of the task percentages, because longer references contribute more characters [7].

> **Illustration with made-up numbers (not a result).** Text A has 1 error in 20 reference characters (5%). Text B has 0 errors in 24 characters (0%). The plain average of the two percentages is 2.5%. The overall CER is $1/44 \approx 2.27\%$.

Overall scores of two models can only be compared fairly if both models were tested on the same tasks.

3. Worked Examples

In the alignment tables below, a dash (—) means that **no character** exists at that position. It is not a real dash character in the text.

3.1 Example 1: `plant` → `blat`

Step	Reference	Prediction	What happened?
1	p	b	Substitution
2	l	l	Match
3	a	a	Match
4	n	—	Deletion
5	t	t	Match

$S = 1$, $D = 1$, $I = 0$, and $N = 5$ (three characters matched).

$$\text{CER} = \frac{1 + 1 + 0}{5} = \frac{2}{5} = 0.4 = 40\%$$

3.2 Example 2: `cat` → `chats`

Step	Reference	Prediction	What happened?
1	c	c	Match
2	—	h	Insertion
3	a	a	Match
4	t	t	Match
5	—	s	Insertion

$S = 0$, $D = 0$, $I = 2$, and $N = 3$ (all three reference characters matched).

$$\text{CER} = \frac{0 + 0 + 2}{3} = \frac{2}{3} \approx 0.6667 \approx 66.67\%$$

Both results were confirmed by our Python function and by JiWER (Section 8) [7].

4. Text Comparison Rules

The CER formula does not say how the text should be prepared before counting. Different tools make different choices [3][4]. We therefore state our rules clearly and apply them in the same way to every model.

Item	Our rule
Capital letters	Keep them. `C` and `c` are different characters.
Punctuation	Keep and count it.
Spaces	Keep all spaces, including spaces at the beginning and the end.
Line breaks	Keep and count them if they occur in the stored text.
Text cleanup	Do not strip spaces, change to lowercase, or change the text in any other way.
Character unit	One Python Unicode code point (explained below).
Empty reference	Reject it with a clear error message.
Empty prediction	Allow it. Every reference character counts as deleted.

These are valid choices for this experiment, not universal requirements of CER [7].

4.1 What counts as one character?

Python stores text as a sequence of **Unicode code points** (numbers that the Unicode standard assigns to characters) [5]. Our code counts code points with Python's `len()` function. A code point is **not always** the same as one symbol that a person sees. The Python documentation gives the example of `ê`: it can be stored as one code point, or as the letter `e` followed by a separate combining accent. Both look the same when printed, but the first has length 1 and the second has length 2 [5].

Our code does not merge such parts into visible symbols. It also does not convert text to one standard form (this conversion is called **Unicode normalization** [5]). This matters for languages that use combining marks. For example, the Khmer syllable *kā* is stored as two code points: the letter KA followed by the vowel sign AA (we checked this with Python's `unicodedata` module). Our simple English tasks are not affected, but scores for other languages must be read with this rule in mind [7].

4.2 Empty references and empty predictions

If the reference is empty, then $N = 0$, and the usual formula would divide by zero. **Our function rejects an empty reference.** An empty prediction is allowed and gives a CER of 100%.

Other tools may choose differently. Starting with version 4.0, JiWER defines its own behaviour for empty references: it no longer requires references to be non-empty. For example, an empty reference with an empty hypothesis gets a CER of 0, and a non-empty hypothesis against an empty reference counts each character as an insertion [4]. Scores that use different empty-reference policies should not be compared directly.

4.3 Making JiWER follow the same rules

By default, JiWER prepares text for CER with a transformation (a preparation step) called `cer_default`. It removes spaces at the beginning and the end of each text and then splits the text into single characters [3]. That would break our rule of keeping all spaces. We therefore replaced the default with a transformation that **only** splits the original text into characters. JiWER counts spaces as characters in its character calculation [2].

```
# No Strip(), lowercase conversion, or punctuation removal.
character_rules = jiwer.Compose([
    jiwer.ReduceToListOfListOfChars(),
])

result = jiwer.process_characters(
    reference,
    prediction,
    reference_transform=character_rules,
    hypothesis_transform=character_rules,
)
```

5. Python Implementation

Our implementation is in the notebook `CER_Evaluation.ipynb` [7]. It contains two main **functions** (named blocks of code that take inputs and return a result):

1. `character_distance` finds the minimum number of character changes.
2. `calculate_cer` checks the inputs and turns the number of changes into CER.

The notebook does not contact any AI service and does not train a model. The model answers were collected by hand and stored in the notebook (Section 6).

5.1 The `character_distance` function

This function finds the minimum number of character changes (Section 2.3). It uses a method called **dynamic programming**.

Dynamic programming in basic English. Dynamic programming is a way to solve a big problem by first solving many small parts of it and **saving** each small answer so that it can be reused. Here, the small parts are questions like: *"What is the cheapest way to turn the first 3 reference characters into the first 2 prediction characters?"* Each answer is built from three neighbouring answers that were already saved:

- **Deletion:** the answer from the row above, plus 1.
- **Insertion:** the answer to the left, plus 1.
- **Substitution or match:** the answer diagonally up and to the left, plus 1 if the two characters differ, or plus 0 if they match.

In the code below, `min()` keeps the cheapest of the three. The code stores only two rows at a time: `previous_row` (the row above) and `current_row` (the row being built). The very last number is the minimum total for the whole texts.

```python
def character_distance(reference, prediction):
    """Return the minimum character edits between two strings."""
    # Creating each prediction prefix from empty text needs insertions.
    previous_row = list(range(len(prediction) + 1))

    for i, reference_char in enumerate(reference, start=1):
        # Removing this reference prefix to make empty text needs i deletions.
        current_row = [i]

        for j, prediction_char in enumerate(prediction, start=1):
            replacement_cost = 0 if reference_char == prediction_char else 1

            deletion = previous_row[j] + 1
            insertion = current_row[j - 1] + 1
            substitution = previous_row[j - 1] + replacement_cost

            current_row.append(min(deletion, insertion, substitution))

        previous_row = current_row

    return previous_row[-1]
```

The table below shows every saved answer for `plant` → `blat`. Rows are reference prefixes (beginnings of the reference), and columns are prediction prefixes. The cell in row "pla" and column "bl", for example, holds the cheapest number of changes that turns `pla` into `bl`. (This table was produced by the same algorithm as the function.)

Reference ↓ / Prediction →	(empty)	b	bl	bla	blat
(empty)	0	1	2	3	4
p	1	1	2	3	4
pl	2	2	1	2	3
pla	3	3	2	1	2
plan	4	4	3	2	2
plant	5	5	4	3	**2**

The bottom-right value is **2**, which matches the hand count in Section 3.1 (one substitution and one deletion).

What the function does not do. It returns only the total number of changes, not separate S, D, and I counts. We use JiWER to display those counts. The method compares each reference character with each prediction character, so the work grows with the product of the two text lengths [6]. This is fast enough for short classroom texts.

5.2 The `calculate_cer` function

```python
def calculate_cer(reference, prediction):
    """Return CER as a ratio; reject non-text inputs and empty references."""
    if not isinstance(reference, str):
        raise TypeError("Reference must be text.")
    if not isinstance(prediction, str):
        raise TypeError("Prediction must be text.")
    if len(reference) == 0:
        raise ValueError("Reference must contain at least one character.")

    errors = character_distance(reference, prediction)
    return errors / len(reference)
```

The function works in three steps:

1. **Check the inputs.** Both values must be text (`str`). `raise` stops the function with a clear error message if a check fails. An empty reference is rejected (Section 4.2). The function never removes or changes any text.
2. **Count the errors** with `character_distance`.
3. **Divide** by the reference length and return a ratio. For `plant` → `blat`, the saved notebook output is `0.4`, displayed as `40.00%` [7].

5.3 How we validated the code

We used two kinds of checks before scoring any model answer:

- **Known-answer checks.** We wrote 13 small text pairs whose correct error counts we worked out by hand. They cover matches, each error type, capital letters, punctuation, spaces, an empty prediction, and a CER above 100%. Python's `assert` statement (a line that stops the program if a condition is false) checks the error count and the CER for every pair.
- **Comparison with JiWER.** JiWER is an existing Python library that calculates CER using minimum edit distance [4]. We ran it on the same 13 pairs using our exact-text rules (Section 4.3), and checked that its total errors and CER agree with our function. Two independent implementations that agree make a coding mistake less likely.

We also checked that invalid inputs are rejected. The results of all checks are in Section 8.

6. Experiment Method

The assignment asks each group to evaluate outputs from at least two LLMs [1]. We tested two chat services by hand.

6.1 Services tested

Service	Label recorded by the student	What we know
Claude	"Sonnet 5.5 Medium"	Recorded as reported by the student. The exact model version was **not independently checked**.
Gemini	"Auto"	An automatic selection mode. It **does not identify a specific fixed model**, so a repeated test may use a different model.

These labels describe the services and settings as the student recorded them. They are not verified model identifiers [7].

6.2 Task and prompt

The task was **spelling correction**: fix the spelling while keeping the wording, capitalization, and punctuation unchanged. Each prompt (the message sent to the chat service) contained this instruction, followed by a blank line and the input sentence:

> Correct the spelling mistakes in the sentence below. Keep the wording, capitalization, and punctuation unchanged except where needed to fix spelling. Return only the corrected sentence, without explanations.

The two tested inputs were:

- **Task 1:** `The cat is sleping.`
- **Task 2:** `I drink watter every day.`

The reference answers were not included in the prompts. The models saw only the instruction and the misspelled sentence.

6.3 Prepared and tested tasks

Five tasks were prepared, but **only Tasks 1 and 2 were tested**. Tasks 3–5 have no collected answers and no scores. They are listed only for transparency and are **not** included in any total.

Task	Input	Reference	Status
1	`The cat is sleping.`	`The cat is sleeping.`	Tested
2	`I drink watter every day.`	`I drink water every day.`	Tested
3	`She goes to schol.`	`She goes to school.`	Not tested
4	`Please close the dor.`	`Please close the door.`	Not tested
5	`We are learning Pythn.`	`We are learning Python.`	Not tested

6.4 Intended procedure

The procedure that the group intended to follow was:

1. Use the same prompt and input for both services.
2. Use a separate new chat for each task. The notebook records that this was the instruction during collection. However, no chat history was stored, so we **cannot independently verify** that this step was followed.
3. Do not include the reference answer in the prompt.
4. Copy each answer exactly, without correcting it or removing extra text on purpose.
5. Score all answers with the same text rules (Section 4).

6.5 How the answers were recorded

The answers were **copied manually** from the chat services and stored in the notebook as **student-reported observations**. Four answers were recorded: two tasks × two services. No program sent requests to the services. The notebook cannot check invisible characters or formatting that might have changed during copying [7].

6.6 Missing information

Information	Status
Exact date of collection	Not recorded
Complete chat settings	Not recorded
Exact Claude model version	Not independently verified (student label only)
Model behind Gemini "Auto"	Not identified
Chat-history evidence of the new-chat procedure	Not stored

These details should be added only if the original chat records confirm them. They limit how well another person can repeat the experiment.

7. Experimental Results

7.1 Collected answers

All values in this section come from the saved outputs of the notebook [7].

Task	Reference	Claude answer	Gemini answer
1	`The cat is sleeping.`	`The cat is sleeping.`	`The cat is sleeping.`
2	`I drink water every day.`	`I drink water every day.`	`I drink water every day.`

The reference for Task 1 has **20** characters (16 letters, 3 spaces, and 1 full stop). The reference for Task 2 has **24** characters (19 letters, 4 spaces, and 1 full stop).

7.2 Score for each answer

Task	Service (recorded label)	S	D	I	Errors	N	CER
1	Claude ("Sonnet 5.5 Medium")	0	0	0	0	20	0.00%
1	Gemini ("Auto")	0	0	0	0	20	0.00%
2	Claude ("Sonnet 5.5 Medium")	0	0	0	0	24	0.00%
2	Gemini ("Auto")	0	0	0	0	24	0.00%

7.3 Overall score for each service

Using the formula from Section 2.7, for each service:

$$\text{Overall CER} = \frac{0 + 0}{20 + 24} = \frac{0}{44} = 0\%$$

Service (recorded label)	Tasks	Total errors	Total N	Overall CER
Claude ("Sonnet 5.5 Medium")	2	0	44	0.00%
Gemini ("Auto")	2	0	44	0.00%

Total errors = all substitutions + deletions + insertions; Total N = all reference characters (20 + 24).

7.4 What the results mean

- Both services returned answers that **matched both references exactly** under our rules, so both have an overall CER of **0%** across 44 reference characters.
- The comparison found **no difference** between the two services on these tasks.
- This does **not** mean that either service is perfect, or that the two services are equally good at all tasks. The sentences were short and simple, each had only one misspelled word, and each service gave only one answer per task.
- Tasks 3–5 were not tested. We do not guess their answers and they are not part of the totals.

8. Code Verification

The checks in this section test **our code**, not the models. The text pairs below are **teaching examples** made up to test the calculation. They are not collected model answers and are kept separate from the experiment in Section 7.

8.1 Software environment

The notebook's saved output reports the environment used [7]:

Item	Value
Python	3.11.15
Conda environment (an isolated set of installed packages)	Py311-TF
JiWER	4.0.0

8.2 Known-answer checks (13 of 13 passed)

Quotation marks are shown so that spaces are visible. `''` means empty text.

#	Reference	Prediction	What it tests	Errors	CER
1	`'cat'`	`'cat'`	Exact match	0	0.00%
2	`'cat'`	`'cut'`	Substitution	1	33.33%
3	`'cat'`	`'ct'`	Deletion	1	33.33%
4	`'cat'`	`'cart'`	Insertion	1	33.33%
5	`'plant'`	`'blat'`	Worked Example 1	2	40.00%
6	`'cat'`	`'chats'`	Worked Example 2	2	66.67%
7	`'cat'`	`''`	Empty prediction	3	100.00%
8	`'a'`	`'abcd'`	CER above 100%	3	300.00%
9	`'cat'`	`'Cat'`	Capital letter counts	1	33.33%
10	`'cat'`	`'cat!'`	Punctuation counts	1	33.33%
11	`'cat'`	`' cat'`	Space at the beginning counts	1	33.33%
12	`'cat'`	`'cat '`	Space at the end counts	1	33.33%
13	`'a cat'`	`'acat'`	Missing inner space counts	1	20.00%

Saved output: **Known-answer checks passed: 13/13** [7].

8.3 Invalid-input checks (4 of 4 passed)

Reference	Prediction	Expected result	Message shown
`''`	`'cat'`	`ValueError`	Reference must contain at least one character.
`''`	`''`	`ValueError`	Reference must contain at least one character.
`123`	`'cat'`	`TypeError`	Reference must be text.
`'cat'`	`None`	`TypeError`	Prediction must be text.

All four planned checks were rejected with the expected error type and message. These messages are intended results, not program failures [7].

8.4 Comparison with JiWER (13 of 13 passed)

For each of the 13 known-answer pairs, JiWER's $S + D + I$ equalled the expected error count, and JiWER's CER equalled our CER (difference smaller than 10^{-12}). Saved output: **JiWER comparison checks passed: 13/13** [7]. JiWER also gave the separate counts for the worked examples: `plant` → `blat` had $S = 1, D = 1, I = 0$ (40.00%), and `cat` → `chats` had $S = 0, D = 0, I = 2$ (66.67%).

8.5 Checks performed while preparing this report

- A **copy** of the notebook was run again from a new, empty Python session in the same environment. All 13 code cells produced the same outputs as the saved notebook. The original notebook was not changed.
- The experiment numbers (20, 24, 44 characters; 0 errors), the worked examples, and the teaching example in Section 9.3 were recalculated separately with JiWER 4.0.0 using the same exact-text rules. All values matched.

8.6 Code checks versus the model experiment

	Code checks (this section)	Model experiment (Section 7)
Purpose	Show that the CER code is correct	Compare two chat services
Source of the text	Small pairs written by the group	Answers copied from Claude and Gemini
Number of cases	13 known-answer + 4 invalid-input	4 answers (2 tasks × 2 services)
Result	All checks passed	Both services: 0% overall CER

9. Strengths and Limitations

9.1 Strengths

- **Easy to explain and calculate:** count the character changes and divide by the reference length.
- **Detects small differences:** one missing letter, one wrong capital letter, or one extra punctuation mark changes the score.
- **No word splitting needed:** the calculation works directly on characters, so we do not need rules for where words start and end.
- **Gives partial credit inside words:** a nearly correct word scores better than a completely wrong one.
- **Well supported by tools:** existing libraries such as JiWER can check our own implementation [2] [4].

9.2 Limitations

- **It does not understand meaning.** CER compares characters, not ideas. A different but equally valid wording can receive a high CER.
- **It does not establish factual correctness.** A very small character change can create an important factual error (see Section 9.3).
- **It depends on a reliable reference.** If the reference is wrong, a correct answer can look wrong.
- **Text rules change the score.** Capital letters, spaces, punctuation, and Unicode handling must be stated. Scores calculated with different rules are not directly comparable [3][4].
- **The character unit matters.** A Python code point is not always one visible symbol [5].
- **Our experiment was very small.** Two easy tasks, with one answer per service per task, cannot establish an overall ranking of the models.
- **Our records are incomplete.** The Claude label was not verified, the model behind Gemini "Auto" was not identified, and the collection date and chat settings were not recorded. This limits repeatability.

9.3 Teaching example: a low CER can hide a wrong fact

> **Teaching example — not a collected model result.** This example was made up to explain a limitation of CER. It is not an additional experiment and does not make any claim about a real product or service.

Text	Value
Reference	`The card is valid for 4 years.`
Prediction	`The card is valid for 5 years.`

The first 22 characters match. Then `4` is replaced by `5`, and the last 7 characters match. So $S = 1$, $D = 0$, $I = 0$, and $N = 30$.

$$\mathrm{CER} = \frac{1}{30} \approx 0.0333 \approx 3.33\%$$

The CER is very low, and the text looks almost correct. Yet the stated fact is **wrong**: the card's validity changed from 4 years to 5 years. A reader who trusts only the CER could miss this mistake. For tasks where facts matter, answers must also be checked for meaning and factual correctness [7].

10. Conclusion

CER measures the smallest number of character substitutions, deletions, and insertions, divided by the number of characters in the reference. A lower CER means a closer match, 0% means an exact match under the chosen rules, and CER can exceed 100% when the prediction contains many extra characters.

Our Python implementation uses dynamic programming to find the minimum number of changes. It passed 13 of 13 known-answer checks, 4 of 4 invalid-input checks, and 13 of 13 comparisons with JiWER configured to the same exact-text rules.

In our small experiment, **both Claude and Gemini matched both references exactly**, with an overall CER of 0% across 44 reference characters each. Because only two easy tasks were tested and the model records are incomplete, **no general winner can be identified**.

When and why to use CER. Use CER when the exact characters of the output matter: reading text from images, turning speech into text, spelling correction, and other tasks where text must be copied or reproduced exactly. Always report the text rules, because they change the score. For open-ended answers, summaries, or facts, CER should be combined with checks of meaning and factual correctness. A future experiment should use more varied tasks, several answers per task, and complete records of the model versions, settings, and collection dates.

References

[1] Course lecturer. *Group Discussion Assignment: Evaluation Methods for Large Language Models (LLMs)*. Advanced Programming course handout. Local file: Activity_12A-Nov-Ch3-ADV_Programming_Friday.pdf.

[2] JiWER. "process" API reference (`process_characters` and `CharacterOutput`). https://jitsi.github.io/jiwer/reference/process/ (accessed 13 November 2026).

[3] JiWER. "transformations" reference (`cer_default`). https://jitsi.github.io/jiwer/reference/transformations/ (accessed 13 November 2026).

[4] JiWER. Project README, including "A note on empty references". https://github.com/jitsi/jiwer#a-note-on-empty-references (accessed 13 November 2026).

[5] Python Software Foundation. "Unicode HOWTO". Python 3 documentation. https://docs.python.org/3/howto/unicode.html (accessed 13 November 2026).

[6] Wagner, R. A., and Fischer, M. J. (1974). The string-to-string correction problem. *Journal of the ACM*, 21(1), 168–173. https://doi.org/10.1145/321796.321811

[7] Group 9. *CER_Evaluation.ipynb*: Python code, collected answers, and saved outputs for this report. Local file: CER_Evaluation.ipynb.